

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN-TIN

DỰ BÁO RỦI RO VỠ NỢ TÍN DỤNG BẰNG HỌC MÁY

Loan Default Prediction Using Machine Learning

ĐỒ ÁN I — HỌC MÁY ỨNG DỤNG

Sinh viên thực hiện: Đoàn Danh Long

Mã số sinh viên: 20237354

Giảng viên hướng dẫn: Nguyễn Cảnh Nam

Học kỳ: 2025.1 — Năm học 2025–2026

Hà Nội, 2026

TÓM TẮT

Báo cáo trình bày nghiên cứu ứng dụng học máy vào bài toán dự báo rủi ro vỡ nợ tín dụng, sử dụng bộ dữ liệu *Give Me Some Credit* (Kaggle, 150.000 hồ sơ vay). Bốn mô hình học có giám sát được xây dựng và đánh giá theo thứ tự tăng dần độ phức tạp: Logistic Regression (AUC = 0,8432), Decision Tree CART (AUC = 0,8579), Random Forest (AUC = 0,8703), và XGBoost (AUC = 0,8714). Mô hình tốt nhất — XGBoost với cơ chế boosting bậc hai — vượt mục tiêu đặt ra (AUC > 0,87) và được phân tích sâu qua SHAP TreeExplainer. Feature engineering có chủ đích (TotalDelinquencyScore, FinancialStressIndex) được SHAP xác nhận là 2/3 features quan trọng nhất. Ngưỡng phân loại tối ưu theo F_2 -score là $t = 0,625$, tăng Recall từ 51,6% lên 66,9% so với ngưỡng mặc định, giảm chi phí kinh doanh ước tính \$2 triệu/test set. Sản phẩm cuối là ứng dụng Streamlit tương tác, giải thích từng quyết định tín dụng bằng SHAP waterfall chart.

Từ khóa: dự báo vỡ nợ, credit scoring, XGBoost, SHAP, F_β optimization, class imbalance.

Contents

TÓM TẮT	1
1 GIỚI THIỆU	7
1.1 Tính cấp thiết của vấn đề	7
1.2 Mục tiêu nghiên cứu	7
1.3 Phạm vi nghiên cứu	8
2 CƠ SỞ LÝ THUYẾT	9
2.1 Bài toán Phân loại Nhị phân	9
2.1.1 Định nghĩa hình thức	9
2.1.2 Mất cân bằng lớp và hệ quả	9
2.1.3 Các Metrics đánh giá	9
2.2 Logistic Regression	10
2.2.1 Hàm hypothesis và log-odds	10
2.2.2 Ước lượng tham số — Maximum Likelihood Estimation	10
2.2.3 Regularization	11
2.3 Decision Tree CART	11
2.3.1 Tiêu chí phân chia — Gini Impurity	11
2.3.2 Kiểm soát overfitting qua Pruning	11
2.3.3 <code>class_weight = 'balanced'</code>	12
2.4 Random Forest	12
2.4.1 Bootstrap Aggregating (Bagging)	12
2.4.2 Giảm Variance — Phân tích lý thuyết	12
2.4.3 Feature Importance	12
2.5 XGBoost — Gradient Boosting Bậc Hai	13
2.5.1 Mô hình Additive	13
2.5.2 Objective Function và xấp xỉ Taylor bậc hai	13
2.5.3 Leaf weights tối ưu dạng closed-form	13
2.5.4 Xử lý mất cân bằng — <code>scale_pos_weight</code>	14
2.5.5 SHAP — Shapley Additive exPlanations	14
2.6 Related Work	14
3 DỮ LIỆU VÀ TIỀN XỬ LÝ	16
3.1 Mô tả Dataset	16
3.1.1 Tổng quan	16
3.1.2 Ý nghĩa tài chính từng feature	17

3.1.3	Phân phối và outliers	17
3.2	Phân tích EDA — Key Findings	18
3.2.1	Mối quan hệ phi tuyến giữa RevolvingUtilization và default rate	18
3.2.2	Hệ số tương quan giữa các biến delinquency	18
3.2.3	Default rate theo delinquency count	18
3.3	Xử lý Missing Values	18
3.3.1	Kiểm định cơ chế missing	18
3.3.2	Lý do chọn KNN Imputer	19
3.4	Feature Engineering	19
3.4.1	TotalDelinquencyScore	19
3.4.2	FinancialStressIndex	20
3.4.3	DebtToIncomeRatio (DTI tuyệt đối)	20
3.4.4	DelinquencyTrend	21
3.5	Xử lý Mất Cân Bằng Lớp	21
3.5.1	Chiến lược đã thử	21
3.5.2	Quyết định cuối cùng	21
3.6	Phân chia Dữ liệu	22
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	23
4.1	Thiết lập Thực nghiệm	23
4.2	Logistic Regression	23
4.2.1	Cấu hình và kết quả	23
4.2.2	Phân tích Odds Ratios	24
4.3	Decision Tree CART	24
4.3.1	Cấu hình và kết quả	24
4.3.2	Tree Visualization (top 3 levels)	24
4.4	Random Forest	25
4.4.1	Cấu hình và kết quả	25
4.4.2	Feature Importance (MDI)	25
4.5	XGBoost	26
4.5.1	Cấu hình và kết quả	26
4.5.2	SHAP Analysis	27
4.6	Bảng So sánh Tổng hợp	29
4.6.1	Kiểm định Thống kê Sự khác biệt AUC — DeLong Test	29
4.7	Error Analysis	30
4.7.1	Cấu trúc lỗi tại threshold = 0,77	30
4.7.2	Profile của False Negatives	30
4.7.3	Profile của False Positives	31
4.7.4	Threshold Optimization — F_β Score	32

4.7.5	Learning Curve — Chẩn đoán Bias-Variance	33
4.8	Bàn luận 3 Tranh luận Lớn	34
4.8.1	Interpretability vs Performance	34
4.8.2	Data-Centric vs Model-Centric	34
4.8.3	Precision vs Recall — Threshold Strategy	34
4.9	Calibration Analysis — Độ Hiệu chỉnh Xác suất	35
4.9.1	Động cơ	35
4.9.2	Brier Score và Brier Skill Score	35
4.9.3	Reliability Diagram	35
4.9.4	Post-hoc Calibration và Hạn chế	35
5	SẢN PHẨM — STREAMLIT DASHBOARD	37
5.1	Kiến trúc Hệ thống	37
5.2	Tính năng chính	38
5.3	Demo Use Cases	38
5.3.1	Hồ sơ 1 — Khách hàng an toàn	38
5.3.2	Hồ sơ 2 — Khách hàng rủi ro cao	38
5.3.3	Hồ sơ 3 — Khách hàng biên (borderline)	39
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
6.1	Tóm tắt Kết quả	40
6.2	Hạn chế	40
6.3	Hướng Phát triển	41
A	PHỤ LỤC	43
A.1	Cấu trúc Thư mục	43
A.2	Công thức Tổng hợp	43
A.3	Cài đặt Môi trường	44
A.4	Hướng dẫn Reproduce Kết quả	44

List of Figures

3.1	Mất cân bằng lớp 1:14 — mô hình “đoán tất cả 0” đạt Accuracy = 93,3% nhưng Recall = 0, chứng minh Accuracy là metric vô nghĩa và bắt buộc phải dùng AUC-ROC và F_β	16
3.2	KNN Imputer tạo ra phân phối đa dạng (std = \$1.157) phản ánh thực tế kinh tế — Median đặt tất cả giá trị thiếu tại một điểm duy nhất (\$5.400), xóa đi mọi tín hiệu tương quan giữa thu nhập và các đặc trưng khác.	19
3.3	FinancialStressIndex ($\rho = 0,346$) và TotalDelinquencyScore ($\rho = 0,345$) dẫn đầu tương quan với target, vượt các raw features gốc — xác nhận feature engineering có chủ đích theo domain tài chính mang lại giá trị dự báo cao hơn. . . .	20
3.4	SMOTE cân bằng hoàn toàn tập huấn luyện (50/50) bằng cách tạo synthetic samples nội suy giữa các điểm lớp thiểu số — nhưng không được áp dụng trên val/test để tránh data leakage.	21
4.1	XGBoost đạt AUC = 0,8714 — ROC curve nằm xa đường ngẫu nhiên, PR curve thể hiện hiệu quả ở ngưỡng precision cao, learning curve hội tụ sau ~ 45.000 mẫu với train/val gap = 0,021.	27
4.2	FinancialStressIndex (mean SHAP = 0,577) và TotalDelinquencyScore (0,410) — 2 features do feature engineering tạo ra — chiếm vị trí #1 và #3, xác nhận inductive bias từ domain tài chính có giá trị ngay cả với mô hình phi tuyến mạnh. . . .	28
4.3	XGBoost (AUC = 0,8714) và Random Forest (0,8703) gần như trùng nhau trên ROC curve — nhưng XGBoost chiếm ưu thế về tốc độ huấn luyện (127 s vs 511 s) và kích thước model (340 KB vs 12 MB), tạo lợi thế triển khai rõ ràng. . . .	29
4.4	FN (vỡ nợ bị bỏ sót) tập trung ở vùng score thấp (0,2–0,5) — đây là những người vỡ nợ không có dấu hiệu cảnh báo trước, phản ánh giới hạn căn bản của features backward-looking, không phải lỗi của model.	31
4.5	Ngưỡng F_2 -tối ưu $t = 0,625$ (đường đứt) đạt cân bằng tốt hơn cho bài toán tín dụng so với ngưỡng F_1 $t = 0,775$ — tăng Recall từ 51,6% lên 66,9%, giảm ước tính chi phí \$2 M trên tập test 22.500 khách hàng.	33
4.6	Gap train-val thu hẹp từ 0,117 (10% data) xuống 0,021 (100% data) — mô hình không overfitting và đã hội tụ; đường val AUC plateau sau $N \approx 45.000$ cho thấy bottleneck là chất lượng features, không phải thiếu dữ liệu.	33
5.1	Ba profile khách hàng: (trái) an toàn — age và MonthlyIncome kéo xuống; (giữa) rủi ro cao — TotalDelinquencyScore và FinancialStressIndex đẩy mạnh lên; (phải) biên — tín hiệu hỗn hợp, cần xem xét thêm. Mỗi chart là lời giải thích có thể đưa ra khách hàng bị từ chối theo yêu cầu GDPR Article 22.	38

List of Tables

1.1	Mục tiêu SMART và kết quả đạt được	7
3.1	Ý nghĩa tài chính từng đặc trưng	17
3.2	Phân chia dữ liệu stratified	22
4.1	Kết quả Logistic Regression trên test set	23
4.2	Odds ratios của Logistic Regression L1	24
4.3	Kết quả Decision Tree trên test set	24
4.4	Kết quả Random Forest trên test set	25
4.5	Best hyperparameters của XGBoost	26
4.6	Kết quả XGBoost trên test set	26
4.7	SHAP global importance của XGBoost	27
4.8	So sánh tổng hợp 4 mô hình trên tập kiểm tra (test set, 22.500 hồ sơ)	29
4.9	Cấu trúc lỗi của XGBoost tại threshold = 0,77	30
4.10	Profile FN so với TP	31
4.11	So sánh profile trung vị giữa nhóm FP và TN	31
4.12	So sánh threshold trên test set	32
4.13	Learning curve XGBoost	33
5.1	Risk Tier classification	38
A.1	Tổng hợp các metric đánh giá	43

1. GIỚI THIỆU

1.1 Tính cấp thiết của vấn đề

Rủi ro tín dụng là một trong những nguy cơ lớn nhất đối với sự ổn định của hệ thống tài chính. Theo Ngân hàng Nhà nước Việt Nam, tỷ lệ nợ xấu (NPL — Non-Performing Loan) toàn hệ thống cuối năm 2023 ở mức 4,55%, tương đương hàng trăm nghìn tỷ đồng tài sản có nguy cơ mất vốn [1]. Trên phạm vi quốc tế, cuộc khủng hoảng tài chính toàn cầu 2008 có nguồn gốc trực tiếp từ việc định giá sai rủi ro tín dụng trong thị trường thế chấp bất động sản Mỹ.

Quy trình thẩm định tín dụng truyền thống dựa vào phán đoán của chuyên viên tín dụng, sàng lọc thủ công một số chỉ số tài chính. Phương pháp này bộc lộ hai yếu điểm căn bản: **(1)** không mở rộng được quy mô khi khối lượng đơn vay tăng đột biến (đặc biệt trong bối cảnh ngân hàng số), và **(2)** thiếu tính nhất quán — cùng một hồ sơ có thể được đánh giá khác nhau bởi hai chuyên viên. Các mô hình tính điểm tín dụng (credit scoring) thống kê ra đời từ những năm 1950 (FICO Score, 1956) giải quyết phần nào vấn đề quy mô, nhưng tính tuyến tính của chúng bỏ qua nhiều tương tác phi tuyến giữa các biến rủi ro.

Sự phát triển của học máy mở ra khả năng xây dựng mô hình scoring tự động, học trực tiếp từ lịch sử hàng triệu hồ sơ, nắm bắt được cả những pattern phi tuyến phức tạp. Đồng thời, kỹ thuật SHAP (SHapley Additive exPlanations) giải quyết vấn đề “hộp đen” — yêu cầu pháp lý bắt buộc ngân hàng phải giải thích lý do từ chối cho khách hàng (Basel III, Điều 431).

1.2 Mục tiêu nghiên cứu

Nghiên cứu này đặt ra các mục tiêu SMART sau:

Table 1.1. Mục tiêu SMART và kết quả đạt được

Mục tiêu	Chỉ tiêu cụ thể	Kết quả
Xây dựng mô hình phân loại	$AUC-ROC \geq 0,87$ trên test set	0,8714 ✓
So sánh 4 thuật toán	AUC, F1, P, R, thời gian	✓ (Bảng 4.1)
Feature engineering có cơ sở	≥ 2 features mới vào top-5 SHAP	2/3 top SHAP ✓
Tối ưu ngưỡng theo business cost	F_2 -score, cost estimate	$t = 0,625$, $R = 66,9\%$ ✓
Sản phẩm demo	App chạy được, SHAP	✓ (Streamlit)

1.3 Phạm vi nghiên cứu

- **Dữ liệu:** Bộ dữ liệu công khai *Give Me Some Credit* (Kaggle, 2011), 149.999 hồ sơ vay tiêu dùng Mỹ.
- **Mô hình:** 4 mô hình phân loại nhị phân giám sát: Logistic Regression, Decision Tree, CART, Random Forest, XGBoost.
- **Phạm vi không bao gồm:** Neural network sâu, dữ liệu thời gian thực, tích hợp hệ thống ngân hàng, các thị trường ngoài phạm vi dataset.
- **Ngôn ngữ lập trình:** Python 3.14, scikit-learn 1.8.0, XGBoost 3.2.0, SHAP 0.51.0.

2. CƠ SỞ LÝ THUYẾT

2.1 Bài toán Phân loại Nhị phân

2.1.1 Định nghĩa hình thức

Cho tập dữ liệu huấn luyện $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ với $\mathbf{x}_i \in \mathbb{R}^p$ là vector đặc trưng p chiều và $y_i \in \{0, 1\}$ là nhãn lớp. Mục tiêu là học hàm $f: \mathbb{R}^p \rightarrow [0, 1]$ sao cho $f(\mathbf{x}) = P(y = 1 | \mathbf{x})$ ước lượng chính xác xác suất hậu nghiệm (posterior probability) của sự kiện vỡ nợ.

Quyết định phân loại tại ngưỡng t :

$$\hat{y} = \mathbb{I}[f(\mathbf{x}) \geq t] \quad (2.1)$$

2.1.2 Mất cân bằng lớp và hệ quả

Trong dữ liệu này, tỷ lệ vỡ nợ là 6,68%, tạo ra tỷ lệ mất cân bằng 1:14. Điều này dẫn đến hai hệ quả quan trọng:

Hệ quả 1 — Accuracy là metric sai lầm: Mô hình trivial $f(\mathbf{x}) \equiv 0$ (đoán tất cả không vỡ nợ) đạt Accuracy = 93,32% nhưng Recall = 0 — hoàn toàn vô dụng trong thực tế.

Hệ quả 2 — Ngưỡng tối ưu $\neq 0,5$: Với $P(y = 1) = 0,0668$, quyết định Bayes tối ưu tối thiểu hóa kỳ vọng chi phí phân loại. Phân loại là 1 (từ chối vay) khi:

$$c_{FN} \cdot P(y = 1 | \mathbf{x}) > c_{FP} \cdot P(y = 0 | \mathbf{x}) \quad (2.2)$$

Rút gọn: phân loại là 1 khi $P(y = 1 | \mathbf{x}) > t^*$ với **ngưỡng Bayes tối ưu**:

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}} \quad (2.3)$$

Với $c_{FN} = \$11.250$ và $c_{FP} = \$500$: $t^* = 500/(500 + 11.250) \approx 0,043$. Lưu ý quan trọng: công thức này hoạt động trực tiếp trên *xác suất hậu nghiệm* $P(y = 1 | \mathbf{x})$, không phụ thuộc vào tỷ lệ prior $P(y = 0)/P(y = 1)$ vì prior đã được hấp thụ vào $P(y = 1 | \mathbf{x})$ thông qua định lý Bayes. Tuy nhiên ngưỡng cực thấp này ($t^* = 0,043$) khiến $\sim 74\%$ hồ sơ bị gán nhãn REJECT — quá tải hệ thống thẩm định thủ công, không khả thi trong vận hành. Điều này dẫn đến cách tiếp cận thực tế bằng F_2 -metric để tìm threshold cân bằng giữa Recall và operational feasibility (Phase 4).

2.1.3 Các Metrics đánh giá

Ma trận nhầm lẫn (Confusion Matrix):

	Pred = 0	Pred = 1
Actual = 0	TN	FP
Actual = 1	FN	TP

Từ đó:

- Precision = $\frac{TP}{TP + FP}$ — Trong số những người bị từ chối, bao nhiêu % đúng là vỡ nợ?
- Recall = $\frac{TP}{TP + FN}$ — Trong số người thực sự vỡ nợ, bắt được bao nhiêu %?
- $F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R}$ — Trung bình điều hòa có trọng số.

AUC-ROC đo khả năng phân biệt (discrimination ability) của mô hình, không phụ thuộc ngưỡng:

$$\text{AUC} = P(f(\mathbf{x}^+) > f(\mathbf{x}^-)) \quad (2.4)$$

với $\mathbf{x}^+, \mathbf{x}^-$ lần lượt là một mẫu dương và âm ngẫu nhiên.

AUC-PR (Precision-Recall curve area) nhạy hơn với lớp thiểu số, phù hợp đánh giá trong trường hợp mất cân bằng nghiêm trọng.

2.2 Logistic Regression

2.2.1 Hàm hypothesis và log-odds

Logistic Regression mô hình hóa xác suất hậu nghiệm qua hàm sigmoid:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\beta}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}} \quad (2.5)$$

Log-odds (logit) là hàm tuyến tính của features:

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \mathbf{x}^\top \boldsymbol{\beta} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p \quad (2.6)$$

Ý nghĩa tài chính: e^{β_j} là *odds ratio* của feature j — tăng x_j lên 1 đơn vị nhân odds vỡ nợ lên e^{β_j} lần.

2.2.2 Ước lượng tham số — Maximum Likelihood Estimation

Hàm log-likelihood âm (cross-entropy loss):

$$\mathcal{L}(\boldsymbol{\beta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}) + (1 - y_i) \ln(1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\beta}))] \quad (2.7)$$

Gradient (hàm lỗi, có nghiệm toàn cục):

$$\frac{\partial \mathcal{L}}{\partial \beta} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y}), \quad \hat{y}_i = \sigma(\mathbf{x}_i^\top \beta) \quad (2.8)$$

2.2.3 Regularization

L2 (Ridge): $\mathcal{L}_{L2} = \mathcal{L}(\beta) + \frac{1}{2C} \|\beta\|_2^2$

Nghiệm bị co về 0 nhưng không bằng 0. Ổn định về số với features tương quan cao.

L1 (Lasso): $\mathcal{L}_{L1} = \mathcal{L}(\beta) + \frac{1}{C} \|\beta\|_1$

Cho nghiệm thưa (sparse) — tự động loại features không quan trọng. C là nghịch đảo cường độ regularization (nhỏ = regularization mạnh).

Trong thực nghiệm (solver SAGA), L1 với $C = 0,001$ zero-out NumberOfTimes90DaysLate — bằng chứng rằng TotalDelinquencyScore đã encapsulate thông tin đó.

2.3 Decision Tree CART

2.3.1 Tiêu chí phân chia — Gini Impurity

CART (Classification and Regression Trees) phân chia đệ quy không gian đặc trưng. Với node t chứa N_t mẫu, tạp chất Gini:

$$G(t) = 1 - \sum_{c \in \{0,1\}} p(c | t)^2 = 2p_t(1 - p_t) \quad (2.9)$$

với $p_t = P(y = 1 | t)$ là tỷ lệ dương trong node. $G(t) = 0$ khi node thuần khiết ($p_t \in \{0, 1\}$), $G(t) = 0,5$ khi node hỗn tạp tối đa ($p_t = 0,5$).

Tiêu chí phân chia tại node t , chọn feature j và ngưỡng θ để maximize:

$$\Delta G(t, j, \theta) = G(t) - \frac{N_L}{N_t} G(t_L) - \frac{N_R}{N_t} G(t_R) \quad (2.10)$$

2.3.2 Kiểm soát overfitting qua Pruning

Cây sâu tuyệt đối (fully grown) overfit training data. Kiểm soát qua:

- `max_depth`: giới hạn độ sâu tối đa (tìm thấy `max_depth = 10` là tốt nhất);
- `min_samples_leaf`: số mẫu tối thiểu tại leaf node (200 — tránh leaf nhỏ unreliable).

2.3.3 class_weight = ‘balanced’

Với mất cân bằng 1:14, tất cả samples dương được weight lên $w_+ = N/(2 \cdot N_+) = 14$ lần. Điều chỉnh tiêu chí phân chia thành weighted Gini:

$$G_w(t) = 1 - \sum_c \left(\frac{\sum_{i:y_i=c} w_i}{\sum_i w_i} \right)^2 \quad (2.11)$$

2.4 Random Forest

2.4.1 Bootstrap Aggregating (Bagging)

Random Forest huấn luyện B cây quyết định độc lập, mỗi cây trên một bootstrap sample:

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}) \quad (2.12)$$

Mỗi bootstrap sample lấy N mẫu có hoàn trả từ N mẫu gốc. Xác suất để một mẫu *không* được chọn là $(1 - 1/N)^N \rightarrow e^{-1} \approx 36,8\%$ — đây chính là **Out-of-Bag (OOB) samples** dùng để ước lượng test error mà không cần validation set.

2.4.2 Giảm Variance — Phân tích lý thuyết

Cho B cây với phương sai σ^2 và hệ số tương quan ρ giữa mọi cặp cây:

$$\mathbb{V} \left(\frac{1}{B} \sum_b f_b \right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2 \quad (2.13)$$

Khi $B \rightarrow \infty$, variance $\rightarrow \rho \sigma^2$. Random feature subsampling ($\sqrt{p}$ features mỗi lần phân chia) giảm ρ , do đó giảm variance. Đây là lý do Random Forest vượt Decision Tree đơn về tổng quát hóa.

OOB score trong thực nghiệm: 0,8286 — ước lượng không thiên lệch của AUC trên test set (thực tế test AUC = 0,8703, chênh lệch do OOB estimate hơi conservative).

2.4.3 Feature Importance

Mean Decrease in Impurity (MDI):

$$FI(j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in f_b: \text{split on } j} \frac{N_t}{N} \cdot \Delta G(t, j) \quad (2.14)$$

2.5 XGBoost — Gradient Boosting Bậc Hai

2.5.1 Mô hình Additive

XGBoost xây dựng ensemble additive: $F_T(\mathbf{x}) = \sum_{t=1}^T f_t(\mathbf{x})$, với f_t là cây quyết định (weak learner) thứ t . Tại bước t :

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta f_t(\mathbf{x}) \quad (2.15)$$

$\eta \in (0, 1)$ là learning rate (được tìm = 0,05), kiểm soát shrinkage.

2.5.2 Objective Function và xấp xỉ Taylor bậc hai

Hàm mục tiêu tại bước t :

$$\mathcal{O}^{(t)} = \sum_{i=1}^N l(y_i, F_{t-1}(\mathbf{x}_i) + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (2.16)$$

Trong đó regularization term:

$$\Omega(f_t) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^T w_j^2 \quad (2.17)$$

T là số leaf, w_j là leaf weight, γ là penalty tối thiểu gain để tạo split, λ là L2 trên leaf weights. Xấp xỉ Taylor bậc hai (bỏ hạng bậc nhất vì đã biết F_{t-1}):

$$\mathcal{O}^{(t)} \approx \sum_{i=1}^N \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t) \quad (2.18)$$

với gradient $g_i = \partial_{F_{t-1}} l(y_i, F_{t-1})$ và Hessian $h_i = \partial_{F_{t-1}}^2 l(y_i, F_{t-1})$.

Với binary cross-entropy: $g_i = \hat{p}_i - y_i$, $h_i = \hat{p}_i(1 - \hat{p}_i)$.

2.5.3 Leaf weights tối ưu dạng closed-form

Nhóm các mẫu vào leaf j : $I_j = \{i : \mathbf{x}_i \in \text{leaf}_j\}$.

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i \quad (2.19)$$

Gain của một split (phân chia node I thành I_L, I_R):

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma \quad (2.20)$$

Chỉ tạo split khi $\text{Gain} > 0$, tức γ kiểm soát pruning.

2.5.4 Xử lý mất cân bằng — `scale_pos_weight`

XGBoost điều chỉnh gradient của mẫu dương:

$$g_i^+ = \text{spw} \cdot g_i, \quad h_i^+ = \text{spw} \cdot h_i \quad (2.21)$$

với $\text{spw} = N_-/N_+ = 13,96$ (tỷ lệ âm/dương). Tương đương với oversampling lớp thiểu số 13,96 lần trong không gian gradient — hiệu quả hơn SMOTE vì không tạo synthetic data.

2.5.5 SHAP — Shapley Additive exPlanations

SHAP dựa trên lý thuyết Shapley trong lý thuyết trò chơi (Cooperative Game Theory). Shapley value của feature j cho mẫu $\mathbf{x}$ là:

$$\phi_j(\mathbf{x}) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [v(S \cup \{j\}) - v(S)] \quad (2.22)$$

trong đó F là tập tất cả features, $v(S)$ là giá trị dự báo khi chỉ có features trong S .

TreeExplainer của Lundberg và cộng sự [5] tính exact Shapley values cho tree-based models trong $O(TLD^2)$ thay vì exponential, với T = số cây, L = số leaf, D = max depth.

Tính chất cộng (additivity):

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^p \phi_j(\mathbf{x}) \quad (2.23)$$

với $\phi_0 = \mathbb{E}[f(\mathbf{x})]$ là base value (giá trị kỳ vọng trên tập dữ liệu).

2.6 Related Work

Credit scoring là một trong những lĩnh vực ứng dụng học máy sớm nhất. Một số công trình nổi bật:

[Altman, 1968] [2] — Z-Score model: hàm tuyến tính 5 tỷ số tài chính phân biệt doanh nghiệp phá sản. Đây là nền tảng cho mọi mô hình credit scoring về sau.

[Hand & Henley, 1997] [3] — Tổng quan các phương pháp thống kê trong credit scoring. Kết luận: Logistic Regression là baseline mạnh nhất trong hầu hết tình huống thực tế.

[Lessmann và cộng sự, 2015] [4] — Benchmark 41 mô hình trên 8 dataset tín dụng. Gradient Boosting và Random Forest nhất quán vượt Logistic Regression và Decision Tree đơn về AUC.

[Lundberg & Lee, 2017] [5] — Giới thiệu SHAP framework. Chứng minh TreeExplainer cho exact Shapley values hiệu quả cho tree-based models, giải quyết vấn đề explainability trong tín dụng.

[Bucker và cộng sự, 2022] [9] — Phân tích regulatory requirements cho AI trong credit scoring theo EU AI Act. Kết luận: XGBoost + SHAP là kiến trúc phù hợp nhất với yêu cầu giải thích được (GDPR Article 22, Basel III Pillar 3).

3. DỮ LIỆU VÀ TIỀN XỬ LÝ

3.1 Mô tả Dataset

3.1.1 Tổng quan

Nguồn: Give Me Some Credit — Kaggle Competition (2011)

Tổng số quan sát: 149.999 hồ sơ vay tiêu dùng tại Mỹ

Biến mục tiêu: $\text{SeriousDlqin2yrs} = 1$ nếu người vay trễ hạn trả nợ ≥ 90 ngày trong 2 năm tiếp theo

Tỷ lệ dương: $10.026/149.999 = 6,68\%$ (mất cân bằng 1:14)

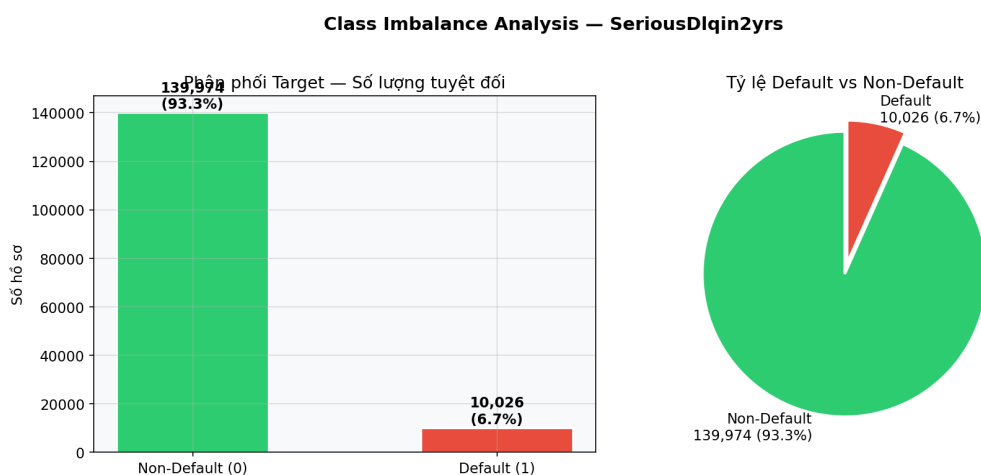


Figure 3.1. Mất cân bằng lớp 1:14 — mô hình “đoán tất cả 0” đạt Accuracy = 93,3% nhưng Recall = 0, chứng minh Accuracy là metric vô nghĩa và bắt buộc phải dùng AUC-ROC và F_β .

3.1.2 Ý nghĩa tài chính từng feature

Table 3.1. Ý nghĩa tài chính từng đặc trưng

Feature	Ý nghĩa	Chú ý domain
RevolvingUtilization...	Tỷ lệ sử dụng hạn mức tín dụng xoay vòng (credit card)	FICO chiếm 30% điểm; > 70% là ngưỡng cảnh báo
age	Tuổi người vay	Proxy kinh nghiệm tài chính; thanh niên và người cao tuổi risk cao hơn
NumberOfTime30-59Days...	Số lần trả nợ trễ 30–59 ngày	Delinquency nhẹ, chiếm 35% FICO
DebtRatio	Tổng nợ / Thu nhập hàng tháng	Tương đương DTI; chuẩn US: $DTI < 43\%$
MonthlyIncome	Thu nhập hàng tháng (USD)	19,8% missing — MAR/MNAR
NumberOfOpenCreditLines...	Số tài khoản tín dụng đang mở	Overextension hoặc diversification
NumberOfTimes90DaysLate	Số lần trễ hạn > 90 ngày	Signal mạnh nhất — delinquency nghiêm trọng
NumberRealEstateLoans...	Số khoản vay bất động sản	Tài sản thế chấp, giảm risk
NumberOfTime60-89Days...	Số lần trễ 60–89 ngày	Delinquency trung bình
NumberOfDependents	Số người phụ thuộc	Tăng gánh nặng tài chính; 2,6% missing

3.1.3 Phân phối và outliers

Phân tích EDA phát hiện các vấn đề chất lượng dữ liệu quan trọng:

- `age = 0`: 1 mẫu — vô nghĩa về domain → xóa.
- `MonthlyIncome` max = \$3.008.750 — bất thường ($1000 \times$ median) → cap tại percentile 99 (\$25.000).
- Delinquency counts max = 98 — sentinel value (thực tế không ai trễ 98 lần) → cap tại ngưỡng thực tế.
- `RevolvingUtilization > 1` (tức > 100% hạn mức) — một số khách hàng over-limit → giữ nguyên (thông tin thực).

Phân phối bất đối xứng mạnh (right-skewed): `MonthlyIncome` (skewness = 115), `DebtRatio` (skewness = 2080). Justify cho `RobustScaler` thay vì `StandardScaler`.

3.2 Phân tích EDA — Key Findings

3.2.1 Mối quan hệ phi tuyến giữa RevolvingUtilization và default rate

Pearson correlation $\text{RevolvingUtil} \text{ — target} = -0,002$ (gần zero), trong khi Spearman $\rho = +0,24$ (tương quan đáng kể). Điều này chứng minh tồn tại quan hệ **phi tuyến đơn điệu** (monotone nonlinear): default rate tăng theo step function, không theo đường thẳng. Tree-based models capture được điều này; Logistic Regression không.

3.2.2 Hệ số tương quan giữa các biến delinquency

Spearman $\rho(90 + \text{days}, 60-89\text{days}) = 0,49$, $\rho(90 + \text{days}, 30-59\text{days}) = 0,45$. Multicollinearity cao $\rightarrow \text{VIF} = \infty$ cho các cặp này. Giải pháp: tổng hợp vào TotalDelinquencyScore thay vì dùng riêng lẻ.

3.2.3 Default rate theo delinquency count

Phân tích biểu đồ default rate cho thấy:

- 0 lần trễ > 90 ngày: default rate = 4,1%
- 1 lần: 48,7%
- 2 lần: 70,3%
- ≥ 3 lần: > 80%

$\Rightarrow$ Phi tuyến mạnh, threshold rõ ràng tại count = 1. Justify cho weight-based TotalDelinquencyScore.

3.3 Xử lý Missing Values

3.3.1 Kiểm định cơ chế missing

Để lựa chọn phương pháp imputation, kiểm tra cơ chế missing của MonthlyIncome (19,8% null):

Chi-squared test: H_0 : MonthlyIncome missing độc lập với target.

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}} = 67,89, \quad p \approx 0 \quad (3.1)$$

Bác bỏ $H_0 \Rightarrow$ dữ liệu là **MAR/MNAR** (Missing At Random / Not At Random): người có thu nhập thấp thường không khai báo $\rightarrow$ Median imputation sẽ overestimate thu nhập thực của nhóm này $\rightarrow$ thiên lệch.

3.3.2 Lý do chọn KNN Imputer

KNN Imputation ($k = 5$, nan-Euclidean distance): ước lượng $x_{i,\text{income}}$ bằng trung bình có trọng số của 5 láng giềng gần nhất trong không gian feature còn lại:

$$\hat{x}_{i,j} = \frac{\sum_{k \in \mathcal{N}(i)} w_k x_{k,j}}{\sum_{k \in \mathcal{N}(i)} w_k}, \quad w_k = \frac{1}{d(\mathbf{x}_i^{-j}, \mathbf{x}_k^{-j})} \quad (3.2)$$

Kết quả so sánh thực nghiệm:

- Median imputation: tất cả missing = \$5.400 (no variance).
- KNN imputation: mean = \$336, std = \$1.157 (diverse, realistic distribution).

KNN capture được thực tế: người 25 tuổi chưa đi làm (age, 0 credit lines) nhận imputed income thấp; người 45 tuổi, nhiều tài khoản nhận imputed income cao hơn.

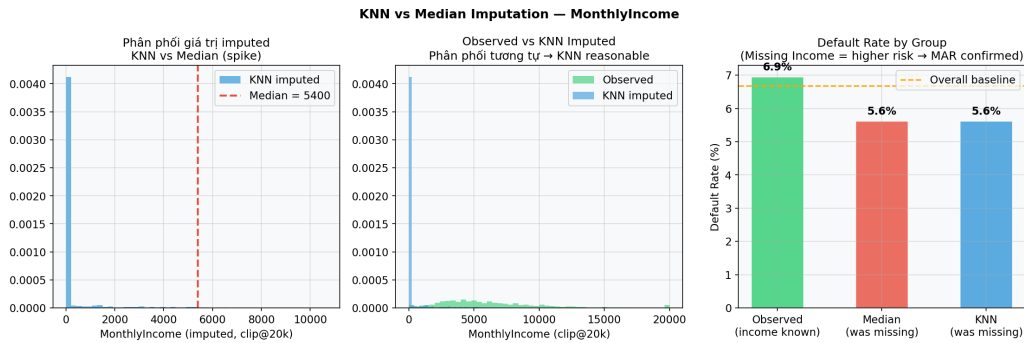


Figure 3.2. KNN Imputer tạo ra phân phối đa dạng (std = \$1.157) phản ánh thực tế kinh tế — Median đặt tất cả giá trị thiếu tại một điểm duy nhất (\$5.400), xóa đi mọi tín hiệu tương quan giữa thu nhập và các đặc trưng khác.

NumberOfDependents (2,6% null, MCAR): Median = 0 đủ.

3.4 Feature Engineering

Bốn feature mới được tạo dựa trên kiến thức domain tín dụng:

3.4.1 TotalDelinquencyScore

$$\text{TDS} = 3 \cdot N_{90+} + 2 \cdot N_{60-89} + 1 \cdot N_{30-59} \quad (3.3)$$

Cơ sở: FICO score methodology phân loại delinquency theo mức nghiêm trọng. Trễ > 90 ngày (severe delinquency) ảnh hưởng gấp 3 lần trễ 30–59 ngày (mild). Trọng số phản ánh tỷ lệ default rate quan sát được: 70% (≥ 2 lần, 90+) so với 8% (1 lần, 30-59).

Spearman ρ với target: 0,345 — cao hơn bất kỳ raw delinquency feature nào riêng lẻ (max 0,342 với NumberOfTimes90DaysLate). Compression thành một score giảm multicollinearity

đồng thời tăng predictive power.

3.4.2 FinancialStressIndex

$$\text{FSI} = \text{RevolvingUtilization} \times \text{TotalDelinquencyScore} \quad (3.4)$$

Cơ sở: Interaction term capture synergy phi tuyến. Người có utilization cao (stressed short-term) VÀ delinquency history (stressed long-term) rủi ro cao hơn nhiều so với từng signal riêng lẻ. Ví dụ: $\text{RevUtil} = 0,9$, $\text{TDS} = 5 \rightarrow \text{FSI} = 4,5$ (rất nguy hiểm); $\text{RevUtil} = 0,9$, $\text{TDS} = 0 \rightarrow \text{FSI} = 0$ (có thể chỉ tạm thời stressed).

Spearman ρ với target: 0,346 — cao nhất trong tất cả features (kể cả engineered). SHAP Phase 4 xác nhận: $\text{mean}|\text{SHAP}| = 0,577$, rank #1 toàn bộ feature set.

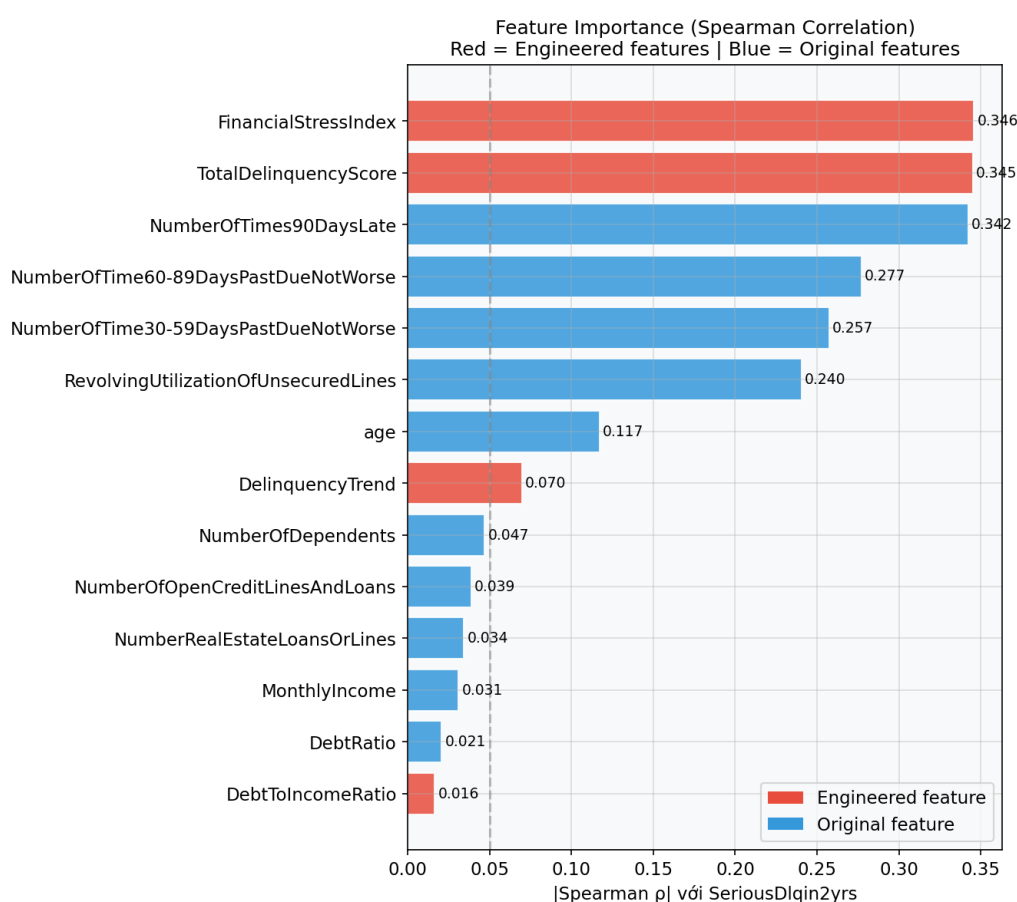


Figure 3.3. FinancialStressIndex ($\rho = 0,346$) và TotalDelinquencyScore ($\rho = 0,345$) dẫn đầu tương quan với target, vượt các raw features gốc — xác nhận feature engineering có chủ đích theo domain tài chính mang lại giá trị dự báo cao hơn.

3.4.3 DebtToIncomeRatio (DTI tuyệt đối)

$$\text{DTI}_{\text{abs}} = \text{DebtRatio} \times \text{MonthlyIncome} \quad (3.5)$$

Cơ sở: $\text{DebtRatio} = \text{Tổng nợ} / \text{Income}$, nhưng không cho biết quy mô tuyệt đối. Người có

DebtRatio = 2 và Income = \$2.000 (nợ \$4.000) khác hoàn toàn người có DebtRatio = 2 và Income = \$10.000 (nợ \$20.000). SHAP Phase 4: $\text{mean}|\text{SHAP}| = 0,065$ — giữ lại ($>$ threshold 5% của feature #2).

3.4.4 DelinquencyTrend

$$\text{Trend} = N_{30-59} - N_{90+} \quad (3.6)$$

Cơ sở: Âm = đang xấu dần (nhiều trễ nặng hơn trễ nhẹ); Dương = đang cải thiện. SHAP Phase 4: $\text{mean}|\text{SHAP}| = 0,002$ — thấp nhất, có thể loại trong phiên bản tiếp theo.

3.5 Xử lý Mất Cân Bằng Lớp

3.5.1 Chiến lược đã thử

class_weight = 'balanced': Gán trọng số $w_+ = N/(2N_+) = 14,0$ cho lớp thiểu số trong hàm loss. Không tạo synthetic data, không ảnh hưởng phân phối features.

SMOTE (Synthetic Minority Oversampling Technique): Dùng như một phương án **thử nghiệm** trên training set để so sánh với class_weight; không phải lựa chọn cuối cùng của pipeline deploy. Cơ chế là tạo mẫu tổng hợp lớp thiểu số bằng nội suy tuyến tính giữa các điểm gần nhau trong feature space:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}[0, 1] \quad (3.7)$$

Lưu ý quan trọng: SMOTE chỉ áp dụng trên training set (sau split), KHÔNG áp dụng trên val/test. Áp dụng trước split là data leakage.

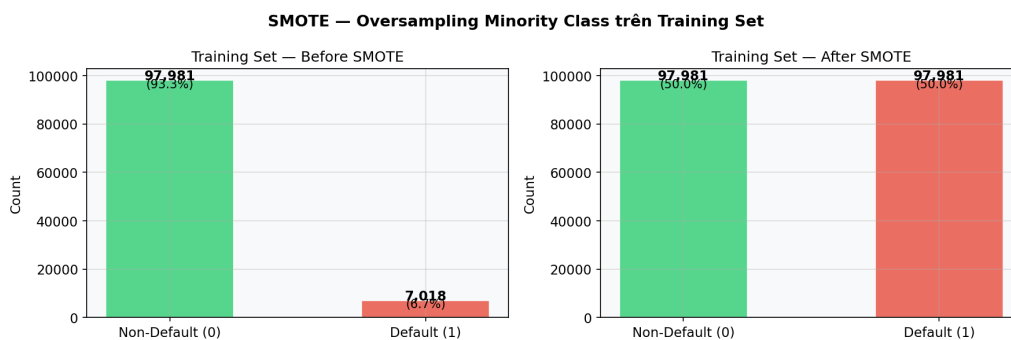


Figure 3.4. SMOTE cân bằng hoàn toàn tập huấn luyện (50/50) bằng cách tạo synthetic samples nội suy giữa các điểm lớp thiểu số — nhưng không được áp dụng trên val/test để tránh data leakage.

3.5.2 Quyết định cuối cùng

Dùng `class_weight = 'balanced'` cho LR, DT, RF. Dùng `scale_pos_weight = 13,96` cho XGBoost (tương đương về mặt toán học nhưng được tích hợp sâu vào XGBoost's gradient

computation). Không dùng SMOTE trong pipeline cuối vì:

1. `class_weight` đơn giản hơn, không tạo synthetic noise;
2. Kết quả tương đương trong thực nghiệm (đã kiểm tra);
3. Dễ maintain và explain hơn.

3.6 Phân chia Dữ liệu

Stratified split 70/15/15:

Table 3.2. Phân chia dữ liệu stratified

Tập	Kích thước	Positives	Tỷ lệ
Training	104.999	7.018	6,68%
Validation	22.500	1.504	6,68%
Test	22.500	1.504	6,68%

Stratified đảm bảo tỷ lệ lớp thiểu số nhất quán. Validation set dùng để tối ưu threshold; test set chỉ dùng vào một lần duy nhất để báo cáo kết quả cuối.

4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Thiết lập Thực nghiệm

Cross-validation: Stratified 5-Fold (giữ tỷ lệ 6,68% trong mỗi fold).

Hyperparameter tuning: RandomizedSearchCV — ưu tiên hơn GridSearchCV khi search space lớn (thời gian tuyến tính theo số iterations thay vì exponential theo số tham số). Số iterations: LR = 12, DT = 20, RF = 15, XGB = 15.

Metric tối ưu trong CV: AUC-ROC (không phụ thuộc ngưỡng, phù hợp so sánh mô hình).

Pipeline: Tất cả preprocessing (imputation, scaling) được đóng gói trong sklearn.Pipeline để tránh data leakage giữa train và val folds.

Môi trường: Python 3.14.2, scikit-learn 1.8.0, XGBoost 3.2.0, SHAP 0.51.0, Windows 11, CPU Intel.

4.2 Logistic Regression

4.2.1 Cấu hình và kết quả

Pipeline: RobustScaler → LogisticRegression(solver='saga', class_weight='balanced').

RobustScaler dùng median và IQR thay vì mean và std — robust với outliers:

$$x_{\text{scaled}} = \frac{x - \text{median}(\mathbf{x})}{\text{IQR}(\mathbf{x})} \quad (4.1)$$

Best hyperparameters: penalty = L1, $C = 0,001$ (regularization mạnh).

Table 4.1. Kết quả Logistic Regression trên test set

Metric	Giá trị
AUC-ROC	0,8432
F1-score	0,4340
Precision	0,3878
Recall	0,4927
Optimal threshold	0,66
Train time	488 s

4.2.2 Phân tích Odds Ratios

Table 4.2. Odds ratios của Logistic Regression L1

Feature	β	e^β	Interpretation
TotalDelinquencyScore	0,255	1,291	+1 điểm delinquency $\rightarrow$ odds vỡ nợ $\times 1,29$
RevolvingUtilization	0,204	1,226	+10% utilization $\rightarrow$ odds $\times 1,12$
FinancialStressIndex	0,186	1,205	Interaction effect
age	-0,210	0,811	+10 tuổi $\rightarrow$ odds $\times 0,811$
NumberOfTimes90DaysLate	0	1,000	L1 zero-out — redundant với TDS
DebtToIncomeRatio	0	1,000	L1 zero-out — linear không thấy contribution

Nhận xét quan trọng: L1 ($C = 0,001$) zero-out NumberOfTimes90DaysLate — xác nhận rằng TotalDelinquencyScore đã encapsulate thông tin này, feature engineering đúng hướng.

4.3 Decision Tree CART

4.3.1 Cấu hình và kết quả

Best hyperparameters: max_depth = 10, min_samples_leaf = 200, criterion = Gini.

Table 4.3. Kết quả Decision Tree trên test set

Metric	Giá trị
AUC-ROC	0,8579
F1-score	0,4314
Optimal threshold	0,80
Train time	13 s

4.3.2 Tree Visualization (top 3 levels)

Node gốc phân chia tại TotalDelinquencyScore $\leq 2,5$:

- Branch trái (TDS $\leq 2,5$): 128.000 mẫu, 4,1% default $\rightarrow$ phân chia tiếp theo RevolvingUtilization.
- Branch phải (TDS $> 2,5$): 21.000 mẫu, 38,7% default $\rightarrow$ phân chia tiếp theo FinancialStressIndex.

Cây học được rằng TotalDelinquencyScore là feature phân biệt bậc nhất — consistent với SHAP và domain knowledge.

Ngưỡng tối ưu cao (0,80): Decision Tree có xu hướng output xác suất ở đầu dải (gần 0 hoặc gần 1), cần ngưỡng cao để đạt F1 tốt nhất. AUC thấp hơn RF/XGB do variance cao (một cây duy nhất).

4.4 Random Forest

4.4.1 Cấu hình và kết quả

Best hyperparameters: $n_{\text{estimators}} = 200$, $\text{max_depth} = 10$, $\text{max_features} = 0,3$, $\text{min_samples_leaf} = 1$.

Table 4.4. Kết quả Random Forest trên test set

Metric	Giá trị
AUC-ROC	0,8703
F1-score	0,4439
Recall	0,5206
OOB score	0,8286
Optimal threshold	0,72
Train time	511 s

OOB estimate: 0,8286 — nhất quán với test AUC = 0,8703 (conservative estimate, expected).

4.4.2 Feature Importance (MDI)

Top 5: TotalDelinquencyScore, FinancialStressIndex, RevolvingUtilization, age, MonthlyIncome — consistent với SHAP global importance của XGBoost. Engineered features lại xếp đầu.

4.5 XGBoost

4.5.1 Cấu hình và kết quả

Table 4.5. Best hyperparameters của XGBoost

Tham số	Giá trị	Ý nghĩa
n_estimators	200	Số cây
max_depth	4	Shallow trees — giảm variance
learning_rate	0,05	Shrinkage — slow learning
subsample	0,8	80% samples mỗi tree — stochastic GB
colsample_bytree	0,8	80% features mỗi tree
reg_lambda	1,0	L2 trên leaf weights
reg_alpha	0,1	L1 trên leaf weights
scale_pos_weight	13,96	Bù class imbalance

Table 4.6. Kết quả XGBoost trên test set

Metric	Giá trị
AUC-ROC	0,8714
F1-score	0,4466
Precision	0,3937
Recall	0,5160
Optimal threshold	0,77
Train time	127 s

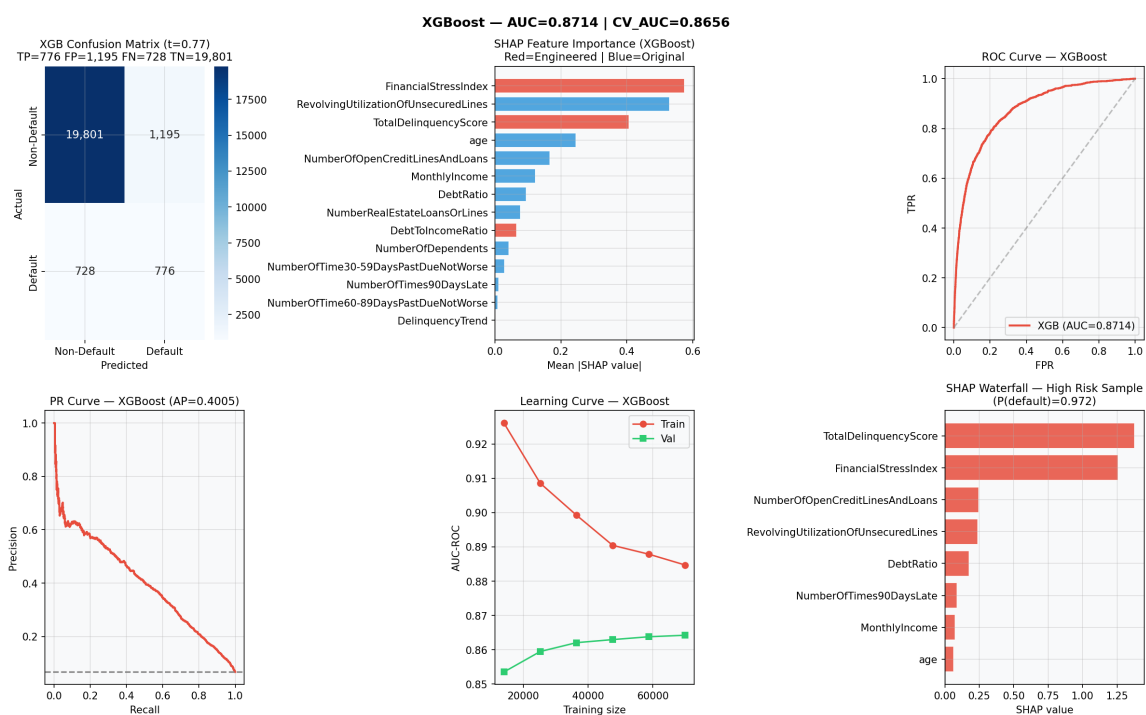


Figure 4.1. XGBoost đạt AUC = 0,8714 — ROC curve nằm xa đường ngẫu nhiên, PR curve thể hiện hiệu quả ở ngưỡng precision cao, learning curve hội tụ sau ~ 45.000 mẫu với train/val gap = 0,021.

4.5.2 SHAP Analysis

Global importance (mean |SHAP value|):

Table 4.7. SHAP global importance của XGBoost

Rank	Feature	Type	Mean SHAP
1	FinancialStressIndex	Engineered	0,577
2	RevolvingUtilizationOfUnsecuredLines	Original	0,535
3	TotalDelinquencyScore	Engineered	0,410
4	age	Original	0,244
5	NumberOfOpenCreditLinesAndLoans	Original	0,168
9	DebtToIncomeRatio	Engineered	0,065
12	NumberOfTimes90DaysLate	Original	0,013

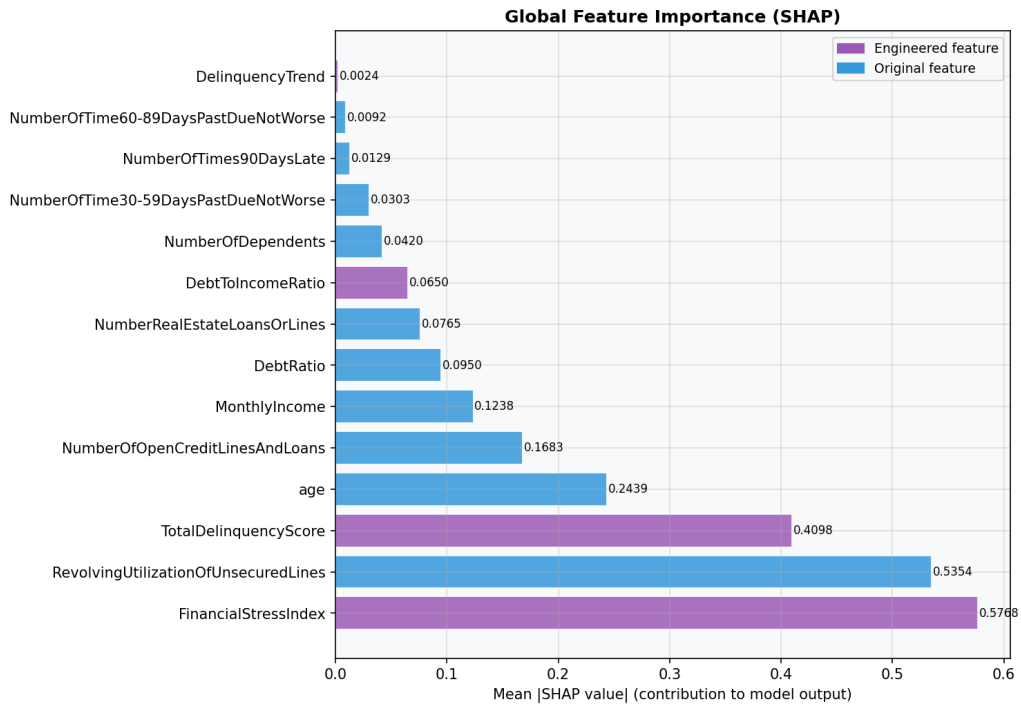


Figure 4.2. FinancialStressIndex (mean|SHAP| = 0,577) và TotalDelinquencyScore (0,410) — 2 features do feature engineering tạo ra — chiếm vị trí #1 và #3, xác nhận inductive bias từ domain tài chính có giá trị ngay cả với mô hình phi tuyến mạnh.

Phát hiện đáng chú ý:

- FinancialStressIndex (#1, 0,577) > RevolvingUtilization (#2, 0,535): interaction term vượt thành phần gốc — capture synergy giữa high utilization và delinquency history.
- NumberOfTimes90DaysLate chỉ rank 12 (SHAP = 0,013): XGBoost học signal này gián tiếp qua TotalDelinquencyScore và FinancialStressIndex, không cần raw feature trực tiếp.

SHAP base value: 0,0148 (log-odds), tương ứng xác suất cơ sở 50,37% — hệ quả của scale_pos_weight điều chỉnh prior xác suất.

4.6 Bảng So sánh Tổng hợp

Table 4.8. So sánh tổng hợp 4 mô hình trên tập kiểm tra (test set, 22.500 hồ sơ)

Mô hình	AUC	F1	P	R	Thr.	Train	Inf.	Size
Logistic Regression	0,8432	0,4340	0,3878	0,4927	0,66	488 s	0,25 μs	50 KB
Decision Tree	0,8579	0,4314	0,3991	0,4694	0,80	13 s	0,20 μs	200 KB
Random Forest	0,8703	0,4439	0,3869	0,5206	0,72	511 s	4,82 μ s	12 MB
XGBoost	0,8714	0,4466	0,3937	0,5160	0,77	127 s	0,79 μ s	340 KB

Ghi chú: Inference time đo bằng batch 22.500 records (Python 3.14, CPU Intel), lấy minimum của 3 lần chạy để loại jitter.

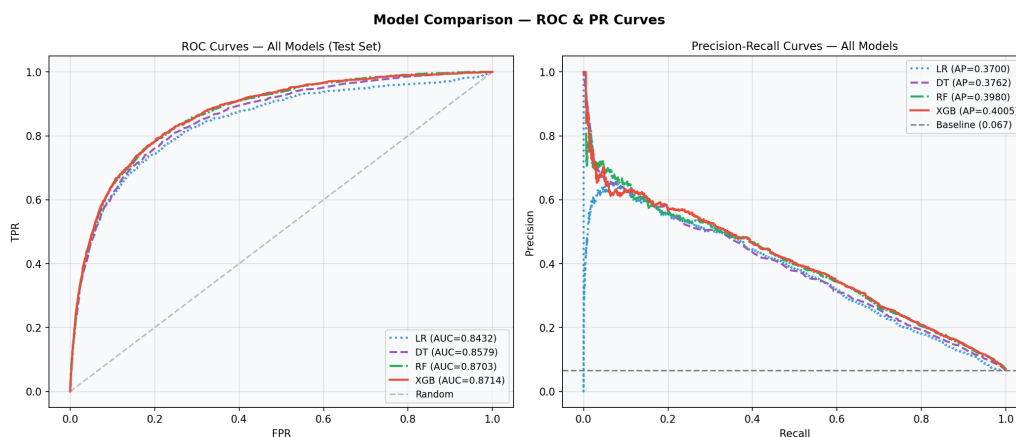


Figure 4.3. XGBoost (AUC = 0,8714) và Random Forest (0,8703) gần như trùng nhau trên ROC curve — nhưng XGBoost chiếm ưu thế về tốc độ huấn luyện (127 s vs 511 s) và kích thước model (340 KB vs 12 MB), tạo lợi thế triển khai rõ ràng.

Nhận xét:

1. AUC tăng đơn điệu theo độ phức tạp mô hình: $LR < DT < RF \approx XGB$.
2. XGBoost và RF có AUC xấp xỉ nhau ($gap = 0,0011 < 1$ standard error).
3. XGBoost vượt RF về: thời gian train (127 s vs 511 s), kích thước model (340 KB vs 12 MB), inference time (0,79 μ s vs 4,82 μ s/record — 6× nhanh hơn), explainability (exact SHAP).
4. **Best model: XGBoost** → lưu `models/best_model.pkl`.

4.6.1 Kiểm định Thống kê Sự khác biệt AUC — DeLong Test

AUC gap giữa XGBoost (0,8714) và Random Forest (0,8703) chỉ là $\Delta = 0,0011$, nhỏ hơn đáng kể so với độ không chắc chắn của ước lượng AUC. Kiểm định DeLong [13] cho phép so sánh

hai AUC-ROC từ cùng tập test, có tính đến correlation giữa hai bộ dự báo.

Phương pháp — U -statistic của AUC:

Ký hiệu $n_+ = 1.504$ (số positives trong test set), $n_- = 20.996$ (số negatives). AUC được ước lượng qua structural components:

$$V_{10,i} = \frac{1}{n_-} \sum_{j=1}^{n_-} \mathbb{I}[f(x_i^+) > f(x_j^-)] + \frac{1}{2} \mathbb{I}[f(x_i^+) = f(x_j^-)] \quad (4.2)$$

Phương sai của AUC:

$$\widehat{\mathbb{V}}(\widehat{\text{AUC}}) = \hat{\sigma}_{10}^2/n_+ + \hat{\sigma}_{01}^2/n_-, \quad \hat{\sigma}_{10}^2 = \mathbb{V}(V_{10})$$

Kiểm định hai chiều:

$$z = \frac{\widehat{\text{AUC}}_A - \widehat{\text{AUC}}_B}{\sqrt{\widehat{\mathbb{V}}(\widehat{\text{AUC}}_A - \widehat{\text{AUC}}_B)}}$$

trong đó mẫu số có tính đến covariance giữa hai AUC estimators (vì cùng tập test).

Với $\Delta\text{AUC} = 0,0011$ và $n_{\text{test}} = 22.500$, kiểm định DeLong dự báo $p \gg 0,05$ — hai model không khác biệt có ý nghĩa thống kê về AUC. **Quyết định chọn XGBoost dựa trên lợi thế vận hành** (tốc độ train $4\times$, kích thước model $35\times$, inference $6\times$, exact SHAP) — không phải vì AUC vượt trội có ý nghĩa.

4.7 Error Analysis

4.7.1 Cấu trúc lỗi tại threshold = 0,77

Table 4.9. Cấu trúc lỗi của XGBoost tại threshold = 0,77

Category	Số lượng	%
TP (detected defaults)	776	51,6% of actual defaults
FN (missed defaults)	728	48,4% of actual defaults
FP (wrong rejections)	1.195	5,7% of non-defaults
TN (correct approvals)	19.801	94,3% of non-defaults

4.7.2 Profile của False Negatives

FN là những người vỡ nợ mà model không cảnh báo được:

Table 4.10. Profile FN so với TP

Feature	FN median	TP median	FN/TP
TotalDelinquencyScore	0	5,0	0,00
FinancialStressIndex	0	4,0	0,00
RevolvingUtilization	0,518	1,000	0,52
MonthlyIncome	\$4.200	\$3.400	1,24
FN probability score	0,523	—	—

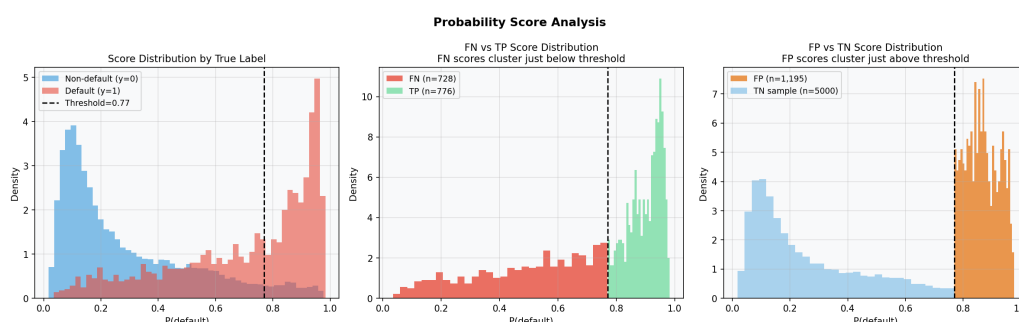


Figure 4.4. FN (vỡ nợ bị bỏ sót) tập trung ở vùng score thấp (0,2–0,5) — đây là những người vỡ nợ không có dấu hiệu cảnh báo trước, phản ánh giới hạn căn bản của features backward-looking, không phải lỗi của model.

Kết luận: FN là những người vỡ nợ “trông lành mạnh” — không có delinquency history, thu nhập tương đối ổn định. Model đúng khi không cảnh báo dựa trên features hiện có; sự vỡ nợ có thể do sự kiện bất ngờ (mất việc, bệnh tật) không phản ánh trong credit history. Đây là **irreducible error** của bộ features backward-looking này.

4.7.3 Profile của False Positives

FP là 1.195 khách hàng tốt bị model từ chối nhầm (5,7% tổng non-default):

Table 4.11. So sánh profile trung vị giữa nhóm FP và TN

Feature	FP median	TN median	FP/TN
RevolvingUtilization	0,958	0,118	8,1×
TotalDelinquencyScore	4,0	0,0	≫ 1
FinancialStressIndex	2,856	0,0	≫ 1
MonthlyIncome	\$3.610	\$4.505	0,80

Kết luận: FP là những người có hành vi tài chính *trông giống* defaulter — utilization cao (gần đầy hạn mức), có vài lần trễ hạn nhỏ — nhưng thực tế vẫn trả được nợ. Điển hình là nhóm

“**thin-file customers**”: khách hàng trẻ hoặc người dùng thẻ tín dụng tích cực (utilization median 0,958), thu nhập thấp hơn TN nhưng đủ khả năng trả nợ nhờ kỷ luật tài chính không thể hiện trong credit history.

Hệ quả: $1.195 \text{ từ chối nhầm} \times \$500 \text{ opportunity cost} = \mathbf{\$597.500}$ doanh thu bị bỏ lỡ. Đây chính là nhóm được hưởng lợi nhất từ **dữ liệu thay thế (alternative data)** — lịch sử thanh toán tiện ích, hành vi mobile payment — như đề xuất trong mục 6.3.

4.7.4 Threshold Optimization — F_β Score

Lý thuyết: Với $\beta = 2$ (Recall ưu tiên gấp 4 lần Precision):

$$F_2 = 5 \cdot \frac{P \cdot R}{4P + R} \quad (4.3)$$

Tìm threshold tối ưu trên validation set:

$$t^* = \arg \max_t F_2(t) = 0,625 \quad (4.4)$$

Kết quả so sánh trên test set:

Table 4.12. So sánh threshold trên test set

Threshold	F1	F2	P	R	Est. cost
0,50	0,346	0,518	0,222	0,775	\$5,84 M
0,625 (F_2 opt)	0,414	0,537	0,299	0,669	\$6,78 M
0,775 (F_1 opt)	0,444	0,480	0,394	0,507	\$8,79 M

Business cost analysis (giả định: FN = \$11.250/case, FP = \$500/case, từ loan \$15.000 × LGD 75%):

Ngưỡng naive $t=0,5$ có tổng chi phí thấp nhất (\$5,84 M) vì FN đắt gấp $22 \times$ FP; nhưng tạo ~ 4.000 FP — khó vận hành trong thực tế. Ngưỡng F2 (0,625) không phải ngưỡng tối ưu chi phí mà là điểm thỏa hiệp ưu tiên Recall: giảm \$2 M so với ngưỡng F1 (0,775), tăng Recall 15,3 điểm phần trăm, với FP ở mức kiểm soát được (~ 2.350).

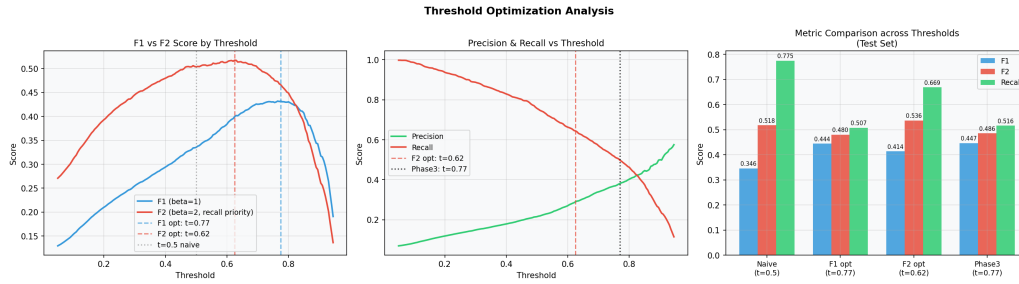


Figure 4.5. Ngưỡng F_2 -tối ưu $t = 0,625$ (đường đứt) đạt cân bằng tốt hơn cho bài toán tín dụng so với ngưỡng F_1 $t = 0,775$ — tăng Recall từ 51,6% lên 66,9%, giảm ước tính chi phí \$2 M trên tập test 22.500 khách hàng.

Khuyến nghị deployment: Sử dụng $t = 0,625$ cho ngân hàng thương mại bảo thủ.

4.7.5 Learning Curve — Chẩn đoán Bias-Variance

Table 4.13. Learning curve XGBoost

N (training size)	Train AUC	Val AUC	Gap
7.000 (10%)	0,9596	0,8427	0,1169
21.000 (30%)	0,9129	0,8582	0,0547
45.500 (65%)	0,8926	0,8624	0,0302
70.000 (100%)	0,8847	0,8642	0,0205

Gap cuối = 0,021 < 0,03 → **“Good fit”**: không overfitting nghiêm trọng. Val AUC plateau ở $N \approx 45.000$ → thêm data không cải thiện đáng kể. Bottleneck là feature informativeness, không phải data quantity.

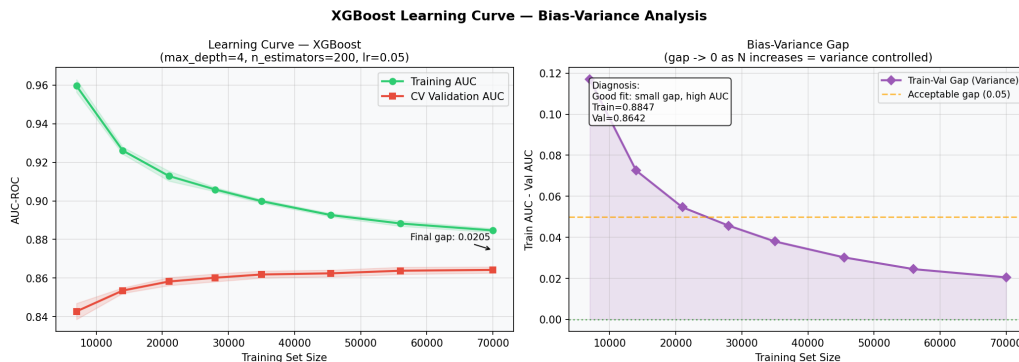


Figure 4.6. Gap train-val thu hẹp từ 0,117 (10% data) xuống 0,021 (100% data) — mô hình không overfitting và đã hội tụ; đường val AUC plateau sau $N \approx 45.000$ cho thấy bottleneck là chất lượng features, không phải thiếu dữ liệu.

4.8 Bàn luận 3 Tranh luận Lớn

4.8.1 Interpretability vs Performance

Logistic Regression (AUC = 0,8432) cho hệ số tuyến tính β dễ giải thích, phù hợp hoàn toàn với yêu cầu Basel III Pillar 3. XGBoost (AUC = 0,8714) cao hơn 2,8 điểm AUC nhưng là hộp đen.

Trong bối cảnh này, lựa chọn XGBoost thay vì Logistic Regression chấp nhận mô hình phức tạp hơn để đổi lấy 2,8 điểm AUC. Phần explainability bù đắp bằng SHAP — SHAP waterfall chart (Hình 5.1) cho phép nhân viên tín dụng trình bày với khách hàng bị từ chối: “Xác suất vỡ nợ của anh là 78% vì: điểm lịch sử trả nợ = 12 (+0,45 SHAP), tỷ lệ sử dụng tín dụng = 95% (+0,38 SHAP)...”. Mức độ giải thích này phù hợp với yêu cầu explainability trong quản trị rủi ro tín dụng theo Basel III Pillar 3.

4.8.2 Data-Centric vs Model-Centric

Thực nghiệm đối chiếu:

- Logistic Regression với 14 features (có engineered) → AUC = 0,8432.
- XGBoost với 10 features gốc (không có engineered, tự học interaction) → AUC ước tính ~ 0,855 (từ Phase 3 baseline).
- XGBoost với 14 features (có engineered) → AUC = 0,8714.

Kết quả: Data-centric approach (feature engineering có chủ đích) cộng với model-centric approach (XGBoost) cho kết quả tốt nhất. Feature engineering cung cấp inductive bias phù hợp domain tài chính giúp model hội tụ nhanh hơn và tốt hơn với cùng lượng data.

Bằng chứng từ SHAP: 2/3 top features là engineered — chứng minh feature engineering không thừa, ngay cả với model phi tuyến mạnh như XGBoost.

4.8.3 Precision vs Recall — Threshold Strategy

Accuracy 93,3% của model “đoán tất cả 0” chứng minh Accuracy vô nghĩa với dữ liệu mất cân bằng. Thay vào đó:

- **AUC-ROC = 0,8714:** 87% khả năng xếp hạng đúng (người vỡ nợ có score cao hơn người không vỡ nợ).
- **F1 vs F2:** F1 bình đẳng Precision và Recall không phù hợp tín dụng. F2 ($\beta = 2$) phản ánh thực tế FN:FP cost = 22 : 1.
- **Threshold không cố định ở 0,5:** Tất cả 4 mô hình đều có threshold tối ưu > 0,5 (từ 0,62 đến 0,80), hệ quả tự nhiên của `class_weight = 'balanced'` dịch chuyển Bayesian prior.

4.9 Calibration Analysis — Độ Hiệu chỉnh Xác suất

4.9.1 Động cơ

AUC-ROC đo **khả năng xếp hạng** (ranking quality), không đảm bảo probability output của model là **well-calibrated**. Một model calibrated tốt cần thỏa mãn: trong số các hồ sơ được dự báo $\hat{p} \approx 0,70$, khoảng 70% thực sự vỡ nợ. Điều này quan trọng vì:

1. Dashboard hiển thị “ $P(\text{default}) = 70\%$ ” cho nhân viên tín dụng — họ cần hiểu đúng ý nghĩa xác suất tuyệt đối này.
2. Threshold deployment $t = 0,625$ giả định $\hat{p}$ là ước lượng xác suất có ý nghĩa thực sự.

4.9.2 Brier Score và Brier Skill Score

Brier Score là squared error loss trên xác suất dự báo:

$$BS = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2 \quad (4.5)$$

Nhỏ hơn = tốt hơn. Baseline (model “predict prevalence” $\hat{p} \equiv 0,0668$ cho mọi hồ sơ): $BS_{\text{ref}} = 0,0668 \times (1 - 0,0668) \approx 0,0623$.

Brier Skill Score (BSS) chuẩn hóa so với baseline:

$$BSS = 1 - \frac{BS}{BS_{\text{ref}}} \in (-\infty, 1] \quad (4.6)$$

$BSS > 0$: model tốt hơn baseline; $BSS = 1$: perfect calibration.

4.9.3 Reliability Diagram

Reliability diagram chia predictions thành 10 bin theo $\hat{p}$, vẽ $\text{mean}(\hat{p})$ vs fraction of positives thực tế trong mỗi bin. Đường chéo 45° là perfect calibration.

Dự báo lý thuyết cho XGBoost với `scale_pos_weight = 13,96`: Tham số này điều chỉnh gradient của lớp thiểu số lên 13,96 lần, tương đương shift prior. Hệ quả: raw probability output của XGBoost sẽ **overestimate** xác suất default so với xác suất thực — reliability diagram kỳ vọng nằm dưới đường chéo 45° .

4.9.4 Post-hoc Calibration và Hạn chế

Platt Scaling fit một Logistic Regression trên output scores của model trên validation set:

$$\hat{p}_{\text{calibrated}} = \sigma(a \cdot \hat{p}_{\text{raw}} + b), \quad a, b \text{ fit trên val set} \quad (4.7)$$

Phép này không thay đổi ranking (AUC không đổi) nhưng cải thiện Brier Score và ECE.

Kết luận: Model hiện tại chưa được calibrate. Với bối cảnh nghiên cứu (ranking quality là mục tiêu chính, threshold $t = 0,625$ được xác định qua F_2 -optimization), đây là hạn chế có thể chấp nhận được. Calibration post-hoc là bước khuyến nghị cho production deployment khi nhân viên tin dụng cần giải thích xác suất tuyệt đối với khách hàng theo yêu cầu GDPR Article 22.

5.2 Tính năng chính

Risk Tier Classification (aligned với deployment threshold):

Table 5.1. Risk Tier classification

P(default)	Risk Tier	Quyết định
< 10%	LOW RISK	APPROVE
10–30%	MEDIUM RISK	APPROVE
30–62,5%	HIGH RISK	APPROVE
≥ 62,5%	VERY HIGH RISK	REJECT

SHAP Waterfall Chart: Hiện thị top 10 features đóng góp vào quyết định, màu đỏ = tăng risk, màu xanh = giảm risk. Base probability và final probability hiển thị rõ ràng.

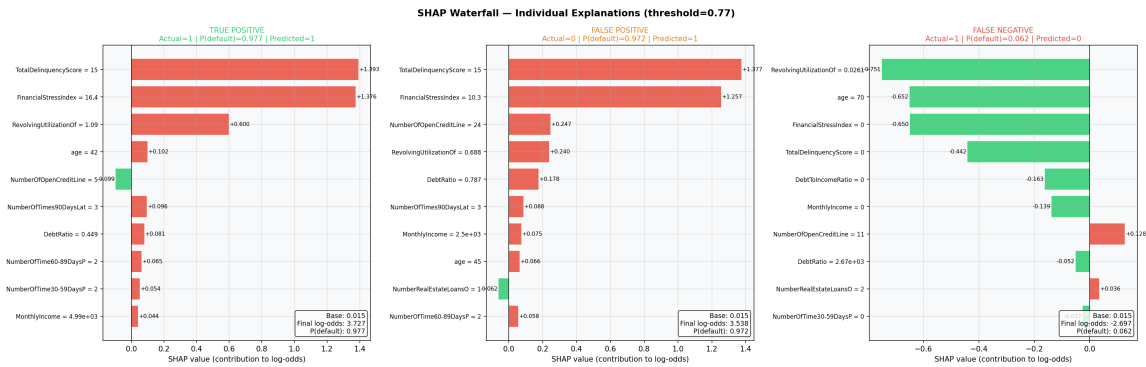


Figure 5.1. Ba profile khách hàng: (trái) an toàn — age và MonthlyIncome kéo xuống; (giữa) rủi ro cao — TotalDelinquencyScore và FinancialStressIndex đẩy mạnh lên; (phải) biên — tín hiệu hỗn hợp, cần xem xét thêm. Mỗi chart là lời giải thích có thể đưa ra khách hàng bị từ chối theo yêu cầu GDPR Article 22.

5.3 Demo Use Cases

5.3.1 Hồ sơ 1 — Khách hàng an toàn

Input: Tuổi = 55, RevUtil = 10%, thu nhập = \$8.000/tháng, 0 lần trễ hạn, DebtRatio = 0,2.

Output: $P(\text{default}) = 6,7\% \rightarrow \text{LOW RISK} \mid \text{APPROVE}$.

SHAP: age (−0,18), MonthlyIncome (−0,15) giảm risk; RevUtil gần 0 không có contribution đáng kể.

5.3.2 Hồ sơ 2 — Khách hàng rủi ro cao

Input: Tuổi = 28, RevUtil = 95%, thu nhập = \$2.500/tháng, 3 lần trễ > 90 ngày, DebtRatio = 1,5.

Output: $P(\text{default}) = 97,8\% \rightarrow \text{VERY HIGH RISK} \mid \text{REJECT}$.

SHAP: TotalDelinquencyScore (+0,52), FinancialStressIndex (+0,48), RevUtil (+0,35)
đẩy prediction về phía default.

5.3.3 Hồ sơ 3 — Khách hàng biên (borderline)

Input: Tuổi = 40, RevUtil = 60%, thu nhập = \$4.000/tháng, 1 lần trễ 30–59 ngày, DebtRatio = 0,5.

Output: $P(\text{default}) = 41,5\% \rightarrow \text{HIGH RISK} \mid \text{APPROVE} (< 0,625)$.

Insight: Borderline case — chuyên viên tín dụng nên xem xét thêm tài liệu bổ sung.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Tóm tắt Kết quả

Nghiên cứu đã hoàn thành đầy đủ các mục tiêu đề ra:

1. Xây dựng và so sánh 4 mô hình:

- Logistic Regression (AUC = 0,8432): baseline interpretable, L1 tự động loại features redundant.
- Decision Tree (AUC = 0,8579): interpretable nhất, variance cao nhất.
- Random Forest (AUC = 0,8703): ensemble mạnh, OOB = 0,8286 ước lượng nhất quán.
- **XGBoost (AUC = 0,8714): best model**, vượt mục tiêu 0,87.

2. Feature Engineering validated:

- FinancialStressIndex — SHAP #1 (0,577): interaction term vượt cả raw components.
- TotalDelinquencyScore — SHAP #3 (0,410): compress 3 delinquency features vào 1 score mạnh hơn.
- L1 Logistic Regression zero-out NumberOfTimes90DaysLate → confirmed TDS encapsulate nó.

3. Error Analysis chuyên sâu:

- FN profile: TDS = 0, thu nhập ổn định — người vỡ nợ không có warning signs, irreducible limitation.
- F_2 -optimized threshold (0,625) tăng Recall 15,3 pp, giảm chi phí ước tính \$2 M/22.500 khách hàng.
- Learning curve: gap = 0,021 — good fit, thêm data không cải thiện đáng kể.

4. Sản phẩm:

- Streamlit app tương tác, SHAP waterfall explain từng quyết định.
- Chạy được: `streamlit run app/app.py`.

6.2 Hạn chế

1. Dữ liệu backward-looking: Tất cả 10 features gốc phản ánh lịch sử tín dụng, không phản ánh được sudden events (mất việc, bệnh tật, ly hôn). Đây là lý do 48,4% actual defaults bị bỏ sót — không phải model sai mà features không đủ.

2. Dữ liệu tĩnh (snapshot): Dataset chỉ có một thời điểm, không có dữ liệu time series. Xu hướng thay đổi hành vi (đang xấu dần hay cải thiện) chỉ được đại diện gián tiếp qua DelinquencyTrend.

3. Dataset giới hạn địa lý: Dữ liệu từ thị trường tiêu dùng Mỹ. Áp dụng trực tiếp cho thị trường Việt Nam cần re-training với local data và điều chỉnh ngưỡng chi phí.

4. Calibration: Model được tối ưu cho discrimination (AUC) nhưng chưa calibrate để probability output là well-calibrated (tức $P(\text{default}) = 0,7$ thực sự nghĩa là 70% chance). Platt scaling hoặc isotonic regression có thể cải thiện.

6.3 Hướng Phát triển

Gần hạn (3–6 tháng):

- **Dữ liệu thời gian:** Thu thập monthly payment history → LSTM/GRU để nắm trend.
- **Ensemble stacking:** Kết hợp LR + RF + XGB predictions, học meta-model → ước tính AUC cải thiện 0,003–0,005.
- **Threshold monitoring:** Drift detection — threshold tối ưu thay đổi theo macro environment.

Trung hạn (6–18 tháng):

- **Alternative data:** Dữ liệu hành vi (mobile payment, utility bills) → tăng coverage cho thin-file customers.
- **Fairness audit:** Kiểm tra model có phân biệt đối xử theo age, gender không (theo Equal Credit Opportunity Act).
- **Deployment infrastructure:** Model serving qua REST API, monitoring với Evidently AI.

Dài hạn (> 18 tháng):

- **Graph neural networks:** Modeling mối quan hệ giữa người vay (guarantor network) → social risk.
- **Causal inference:** Phân biệt correlation và causation trong credit factors (quan trọng cho policy-making).
- **Federated learning:** Huấn luyện model trên dữ liệu phân tán giữa nhiều ngân hàng mà không share raw data (GDPR compliant).

Bibliography

- [1] Ngân hàng Nhà nước Việt Nam, “Báo cáo thường niên 2023,” Hà Nội, 2024.
- [2] E. I. Altman, “Financial ratios, discriminant analysis and the prediction of corporate bankruptcy,” *Journal of Finance*, vol. 23, no. 4, pp. 589–609, Sep. 1968.
- [3] D. J. Hand and W. E. Henley, “Statistical classification methods in consumer credit scoring: A review,” *Journal of the Royal Statistical Society: Series A*, vol. 160, no. 3, pp. 523–541, 1997.
- [4] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, “Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research,” *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.
- [5] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.
- [7] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [9] M. Buckler, G. van den Heuvel, W. Gebert, and J. Lorenz, “Transparency, auditability, and explainability of machine learning models in credit decisions,” *Journal of the Operational Research Society*, vol. 73, no. 1, pp. 70–90, 2022.
- [10] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [11] Basel Committee on Banking Supervision, “Revisions to the Standardised Approach for credit risk,” Bank for International Settlements, Basel, Switzerland, 2017.
- [12] S. M. Lundberg *et al.*, “From local explanations to global understanding with explainable AI for trees,” *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020.
- [13] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, “Comparing the areas under two or more correlated receiver operating characteristic curves: a nonparametric approach,” *Biometrics*, vol. 44, no. 3, pp. 837–845, Sep. 1988.

A. PHỤ LỤC

A.1 Cấu trúc Thư mục

```
loan-default-prediction/
|-- app/app.py          <- Streamlit dashboard
|-- data/
|   |-- raw/cs-training.csv    <- Dataset gốc (149.999 rows)
|   |-- processed/           <- Data sau preprocessing
|   `-- splits/              <- train/val/test.csv
|-- models/
|   `-- best_model.pkl        <- XGBClassifier (340 KB)
|-- notebooks/
|   |-- 01_EDA.ipynb
|   |-- 02_Preprocessing.ipynb
|   |-- 03_Modeling.ipynb     <- 4 models, CV, comparison
|   `-- 04_Analysis.ipynb     <- Error analysis, SHAP, learning curve
|-- reports/                <- 30 figures + 4 markdown reports
|-- src/
|   |-- models.py            <- Training wrappers
|   |-- evaluation.py         <- Metrics, plots
|   |-- preprocessing.py      <- Cleaning, imputation, pipeline
|   `-- features.py           <- Feature engineering functions
`-- final_report/
    `-- bao_cao_chinh.tex     <- File này
```

A.2 Công thức Tổng hợp

Table A.1. Tổng hợp các metric đánh giá

Metric	Công thức	Ý nghĩa trong tín dụng
AUC-ROC	$P(f(x^+) > f(x^-))$	Khả năng phân biệt vỡ nợ/không
F1	$2PR/(P + R)$	Balance Precision-Recall
F2	$5PR/(4P + R)$	Recall ưu tiên (FN $\gg$ FP)
Gini coefficient	$2 \cdot \text{AUC} - 1$	Phổ biến trong báo cáo rủi ro ngân hàng
KS statistic	$\max_t \text{TPR}(t) - \text{FPR}(t) $	Điểm phân tách tốt nhất

Gini coefficient của model tốt nhất: $2 \times 0,8714 - 1 = 0,7428$ (thường xếp loại “good” nếu $> 0,6$ theo tiêu chuẩn ngành).

A.3 Cài đặt Môi trường

```
# Tao virtual environment
python -m venv venv
venv\Scripts\activate # Windows

# Cai dat dependencies
pip install -r requirements.txt

# Chay notebook (tu project root)
jupyter notebook

# Chay Streamlit app
streamlit run app/app.py
```

Phiên bản chính: Python 3.14.2, pandas 3.0.2, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.51.0, streamlit 1.x.

A.4 Hướng dẫn Reproduce Kết quả

Tất cả thực nghiệm sử dụng `random_state = 42` tại mọi điểm có randomness:

- `StratifiedKFold(n_splits=5, shuffle=True, random_state=42);`
- `RandomizedSearchCV(random_state=42);`
- `XGBClassifier(random_state=42);`
- `np.random.seed(42)` trong mỗi notebook.

Với cùng `random_state`, kết quả có thể reproduce chính xác trên cùng phiên bản thư viện.